



OPERATING SYSTEM CONCEPTS

Chapter 4. Threads

A/Prof. Kai Dong

dk@seu.edu.cn

School of Computer Science and Engineering,
Southeast University

April 9, 2024



Contents

- 1 Warm-up
- 2 Overview
- 3 Multicore Programming
- 4 Multithreading Models
- 5 Thread Libraries
- 6 Implicit Threading
- 7 Threading Issues



Warm-up

Thread — A New Abstraction

- Why we need this abstraction?
- Think of: Some programs have more than one point of execution (i.e., multiple PCs).
 - Example #1: adding two large arrays together
 - How to speed up execution?
 - Example #2: some elements need input
 - How to speed up execution?
- Why multiple threads, not multiple processes?
 - We are now detailing the reasons.



Objectives

- To introduce the notion of a thread — a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems
- To discuss the APIs for the Pthreads, Windows, and Java thread libraries
- To explore several strategies that provide implicit threading
- To examine issues related to multithreaded programming
- To cover operating system support for threads in Windows and Linux



Contents

- 1 Warm-up
- 2 Overview**
- 3 Multicore Programming
- 4 Multithreading Models
- 5 Thread Libraries
- 6 Implicit Threading
- 7 Threading Issues



Overview

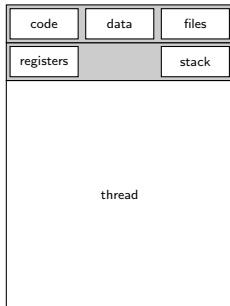
Motivation

- Most modern applications are multithreaded
 - Threads run within application
- Multiple tasks with the application can be implemented by separate threads
 - An example: word processor
 - Update display
 - Fetch data
 - Spell checking
- A single application may be required to perform several similar tasks.
 - Why not multiple processes?
- Multithreaded remote procedure call (RPC) server can serve several concurrent requests.
- Kernels are generally multithreaded.

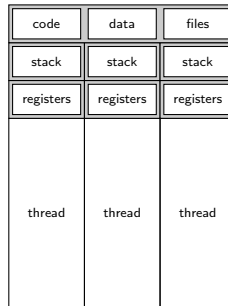


Overview

Single and Multithreaded Processes



Single-Threaded Process



Multi-Threaded Process



Overview

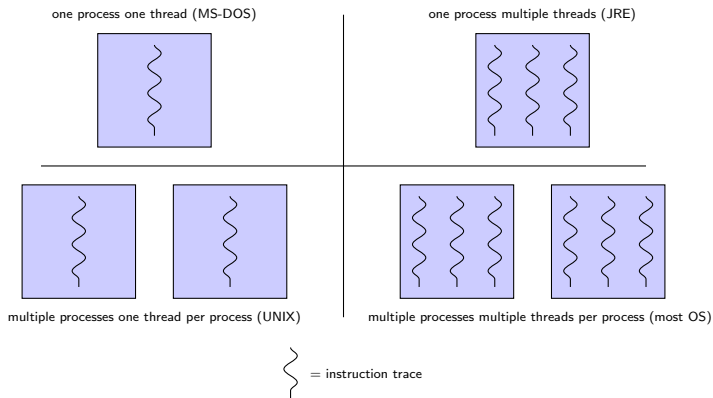
Benefits

- **Responsiveness** — may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** — threads share resources of process, easier than shared memory or message passing
- **Economy** — cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** — process can take advantage of multiprocessor architectures



Overview

Multithreading





Contents

- 1 Warm-up
- 2 Overview
- 3 Multicore Programming**
- 4 Multithreading Models
- 5 Thread Libraries
- 6 Implicit Threading
- 7 Threading Issues



Multicore Programming

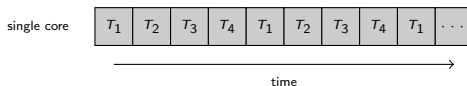
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
 - Dividing activities
 - Balance
 - Data splitting
 - Data dependency
 - Testing and debugging
- **Parallelism** implies a system can perform more than one task simultaneously
- **Concurrency** supports more than one task making progress
 - Single processor / core, scheduler providing concurrency



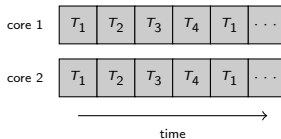
Multicore Programming

Concurrency Vs. Parallelism

- **Concurrent** execution on single-core system:



- **Parallelism** on a multi-core system:





Multicore Programming

- Types of parallelism
 - **Data parallelism** — distributes subsets of the same data across multiple cores, same operation on each
 - **Task parallelism** — distributing threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
 - CPUs have cores as well as hardware threads
 - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core
- How much performance can we gain from adding additional cores? Suppose we have N processing cores.



Multicore Programming

Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches $1/S$
- Serial portion of an application has disproportionate effect on performance gained by adding additional cores



Contents

- 1 Warm-up
- 2 Overview
- 3 Multicore Programming
- 4 Multithreading Models**
- 5 Thread Libraries
- 6 Implicit Threading
- 7 Threading Issues



Multithreading Models

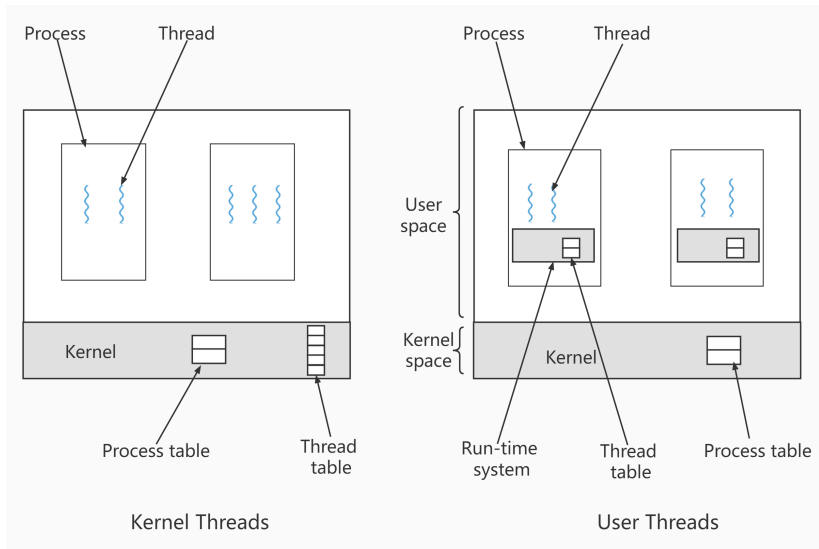
User Threads and Kernel Threads

- Who/How to provide support for threads?
- **User threads** — management done by user-level threads library
- Three primary thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples — virtually all general purpose operating systems, including:
 - Windows
 - Solaris
 - Linux
 - Tru64 UNIX
 - Mac OS X



Multithreading Models

Implementing Threads in Kernel/User Space





Multithreading Models

Motivating Kernel/User Threads

- Why we need user threads?
 - There is no kernel intervention, and, hence, user threads are usually more efficient.
- Why user threads are not enough?
 - Unfortunately, since the kernel only recognizes the containing process (of the threads), if one thread is blocked, every other threads of the same process are also blocked because the containing process is blocked.
- Why we need kernel threads?
 - However, blocking one thread will not cause other threads of the same process to block. The kernel simply runs other threads.
 - In a multiprocessor environment, the kernel can schedule threads on different processors
- Why kernel threads are not enough?
 - Kernel threads are usually slower than the user threads.



Multithreading Models

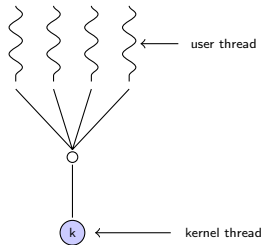
- Many-to-One
- One-to-One
- Many-to-Many



Multithreading Models

Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system
- Used on systems that do not support kernel threads.
- Few systems currently use this model

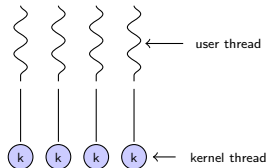




Multithreading Models

One-to-One

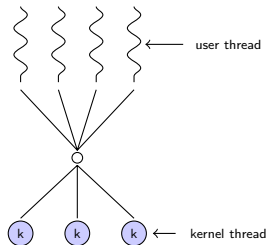
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead



Multithreading Models

Many-to-Many

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads

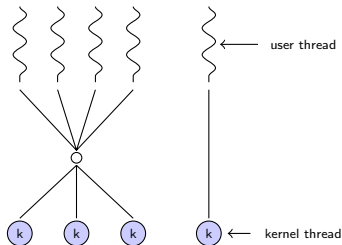




Multithreading Models

Two-Level Model

- Similar to M:M, except that it allows a user thread to be bound to kernel thread





Multithreading Models

Aside

- Among the three models, which model(s) are general in modern systems?
 - M:M for server
 - 1:1 for PC
 - To improve LinuxThreads, two competing projects were started to develop a replacement; NGPT (Next Generation POSIX Threads from IBM) and NPTL (Native POSIX Thread Library from Red Hat). NPTL won out and is today shipped with the vast majority of Linux systems
 - NGPT M:M
 - NPTL 1:1



Contents

- 1 Warm-up
- 2 Overview
- 3 Multicore Programming
- 4 Multithreading Models
- 5 Thread Libraries**
- 6 Implicit Threading
- 7 Threading Issues



Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS
- Three main thread libraries
 - **POSIX Pthreads**
 - Windows
 - Java



Thread Libraries

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- [Specification](#), not [implementation](#)
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)



Thread Libraries

Pthread Creation

```
1 #include <pthread.h>
2 int
3 pthread_create( pthread_t *      thread ,
4                const pthread_attr_t *attr ,
5                void *          (*start_routine) (void*),
6                void *          arg );
```

- ① Which thread is to be created?
 - A pointer to structure of this thread.
- ② What kind of thread is it?
 - The stack size and priority information, etc.
- ③ How to run the thread?
 - The thread entry function (function pointer in C).
- ④ With what arguments?
 - arg.



Thread Libraries

Pthread Creation — What is the Output?

```
1  #include <pthread.h>
2
3  typedef struct __myarg_t {
4      int a;
5      int b;
6  } myarg_t;
7
8  void *mythread(void *arg) {
9      myarg_t *m = (myarg_t *) arg;
10     printf("%d_%d\n", m->a, m->b);
11     return NULL;
12 }
13
14 int main(int argc, char *argv[]) {
15     pthread_t p;
16     int rc;
17     myarg_t args;
18     args.a = 10;
19     args.b = 20;
20     rc = pthread_create(&p, NULL, mythread, &args);
21     ...
22 }
```



Thread Libraries

Pthread Completion

```
1 #include <pthread.h>
2 int
3 pthread_join( pthread_t *    thread ,
4               void **       value_ptr );
```

- ① Which thread is to be completed?
 - A pointer to structure of this thread.
- ② What is the return value?
 - void **value_ptr.



Thread Libraries

Pthread Completion — What is the Output?

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <assert.h>
4  #include <stdlib.h>
5
6  typedef struct __myarg_t {
7      int a;
8      int b;
9  } myarg_t;
10
11 typedef struct __myret_t {
12     int x;
13     int y;
14 } myret_t;
15
16 void *mythread(void *arg) {
17     myarg_t *m = (myarg_t *) arg;
18     printf("%d_%d\n", m->a, m->b);
19     myret_t *r = Malloc(sizeof(myret_t));
20     r->x = 1;
21     r->y = 2;
22     return (void *) r;
23 }

```

```

1  int main(int argc, char *argv[]) {
2      int rc;
3      pthread_t p;
4      myret_t *m;
5      myarg_t args;
6      args->a = 10;
7      args->b = 20;
8      Pthread_create(&p, NULL, mythread, &
9                     args);
9      Pthread_join(p, (void **) &m);
10     printf("returned %d_%d\n", m->x, m->y);
11     return 0;
12 }

```



```

1 #include <stdio.h>
2 #include <pthread.h>
3 #include <assert.h>
4 #include <stdlib.h>
5
6 int tempindata = 0;
7 typedef struct __myarg_t {
8     int a, b;
9 } myarg_t;
10 typedef struct __myret_t {
11     int x, y;
12 } myret_t;
13
14 void *mythread(void *arg) {
15     printf("Add._of_&arg_(inmythread):_%p\n",
16           &arg);
17     printf("Add._of_arg_(inmythread):_%p\n",
18           arg);
19     int tempinthread = 0;
20     printf("Add._of_tempinthread:_%p\n", &
21           tempinthread);
22     myarg_t *m = (myarg_t *) arg;
23     printf("%d_%d\n", m->a, m->b);
24     myret_t *r = (myret_t*)(malloc(sizeof(
25         myret_t)));
26     r->x = 1;
27     r->y = 2;
28     return (void *) r;
29 }

```

```

1 int main(int argc, char *argv[]) {
2     int tempinstack = 0;
3     int *tempinheap = (int*)(malloc(sizeof(
4         int)));
5
6     *tempinheap = 0;
7     printf("Add._of_tempindata:_%p\n", &
8           tempindata);
9     printf("Add._of_tempinstack:_%p\n", &
10           tempinstack);
11     printf("Add._of_tempinheap:_%p\n",
12           tempinheap);
13
14     int rc;
15     pthread_t p;
16     myret_t *m;
17     myarg_t args;
18     args.a = 10;
19     args.b = 20;
20     printf("Add._of_&args_(inmain):_%p\n",
21           &args);
22
23     pthread_create(&p, NULL, mythread, &
24                   args);
25     pthread_join(p, (void **) &m);
26     printf("returned_%d_%d\n", m->x, m->y);
27     return 0;
28 }

```



Thread Libraries

Try It Yourself

```
1 prompt> gcc -o kai kai.c -pthread -Wall
2 prompt> ./kai
3 Add. of tempindata: 0x56385e6ee014
4 Add. of tempinstack: 0x7ffd0075d194
5 Add. of tempinheap: 0x563860287260
6 Add. of &args (inmain): 0x7ffd0075d1b0
7 Add. of &arg (inmythread): 0x7fa38e260ec8
8 Add. of arg (inmythread): 0x7ffd0075d1b0
9 Add. of tempinthread: 0x7fa38e260ed4
10 20
11 returned 1 2
```



Thread Libraries

Windows Multithreaded C Program

```

1  #include <windows.h>
2  #include <stdio.h>
3  DWORD Sum;
4  DWORD WINAPI Summation(LPVOID Param) {
5      DWORD Upper = *(DWORD *) Param;
6      for (DWORD i = 0; i <= Upper; i++)
7          Sum+=i;
8      return 0;
9  }
10
11 int main(int argc, char *argv[]) {
12     DWORD ThreadId;
13     HANDLE ThreadHandle;
14     int Param;
15     if (argc != 2) {
16         fprintf(stderr, "argc!=2\n");
17         return -1;
18     }
19     Param = atoi(argv[1]);
20     if (Param < 0) {
21         fprintf(stderr, "Param<0\n");
22         return -1;
23     }

```

```

1  /* create the thread */
2  ThreadHandle = CreateThread(
3      NULL, /* default security attributes */
4      0, /* default stack size */
5      Summation, /* thread function */
6      &Param, /* parameter to thread function */
7      /*
8       * default creation flags */
9      &ThreadId); /* returns the thread
10                  identifier */
11
12 if (ThreadHandle != NULL) {
13     /* now wait for the thread to finish */
14     WaitForSingleObject(ThreadHandle,
15                         INFINITE);
16 }
17
18 /* close the thread handle */
19 CloseHandle(ThreadHandle);
20
21 printf("sum=%d\n", Sum);
22 }
23 }

```



Thread Libraries

Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- Java threads may be created by:

```
1 public interface Runnable {  
2     public abstract void run();  
3 }
```

- Extending Thread class
- Implementing the Runnable interface



Thread Libraries

Java Multithreaded Program

```
1  class Sum {  
2      private int sum;  
3      public int getSum() {  
4          return sum;  
5      }  
6      public void setSum(int sum) {  
7          this.sum = sum;  
8      }  
9  }  
10 class Summation implements Runnable {  
11     private int upper;  
12     private Sum sumValue;  
13     public Summation(int upper, Sum sumValue) {  
14         this.upper = upper;  
15         this.sumValue = sumValue;  
16     }  
17     public void run() {  
18         int sum = 0;  
19         for (int i = 0; i <= upper; i++)  
20             sum += i;  
21         sumValue.setSum(sum);  
22     }  
23 }
```



Thread Libraries

Java Multithreaded Program (contd.)

```
1 public class Driver {
2     public static void main(String[] args) {
3         if (args.length > 0) {
4             if (Integer.parseInt(args[0]) < 0)
5                 System.err.println(args[0] + "must_be_>=0.");
6             else {
7                 Sum sumObject = new Sum();
8                 int upper = Integer.parseInt(args[0]);
9                 Thread thrd = new Thread(new Summation(upper, sumObject));
10                thrd.start();
11                try {
12                    thrd.join();
13                    System.out.println("The_sum_of_" + upper + "_is_" + sumObject.getSum());
14                } catch (InterruptedException ie) {}
15            }
16        }
17        else
18            System.err.println(" Usage: _Summation<integer_value>");
19    }
20 }
```



In Class Exercise

What are Possible Outputs?

```
1  /* kai.c */
2  #include <stdio.h>
3  #include <pthread.h>
4  void *helloFunc(void *ptr) {
5      int *data;
6      data = (int *) ptr;
7      printf("I'm_Thread_%d\n", *data);
8      return (void *) data;
9  }
10 int main(int argc, char *argv[]) {
11     pthread_t hThread[4];
12     int *rvals[4];
13     for (int i = 0; i < 4; i++)
14         pthread_create(&hThread[i], NULL, helloFunc, (void *) &i);
15     for (int i = 0; i < 4; i++) {
16         pthread_join(hThread[i], (void **) &rvals[i]);
17         //printf("Thread %d returns %d\n", i, *rvals[i])
18     }
19     return 0;
20 }
```

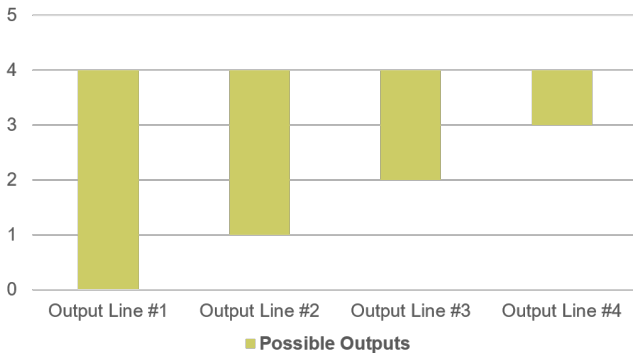
```
1 prompt> gcc -o kai kai.c -pthread -Wall
2 prompt> ./kai
```



In Class Exercise

Key

Any possible order of a non-decreasing sequence satisfying ...





In Class Exercise

More Details: Efficient Use of Memory

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <stdlib.h>
4  void *helloFunc(void *ptr) {
5      int *data = malloc(sizeof(int));
6      data = (int *) ptr;
7      printf("I'm Thread_%d\n", *data);
8      return (void *) data;
9  }
10 int main(int argc, char *argv[]) {
11     pthread_t hThread[4];
12     int pvals[4];
13     int j;
14     for (j = 0; j < 4; j++)
15         pvals[j] = j;
16     for (int i = 0; i < 4; i++)
17         pthread_create(&hThread[i], NULL, helloFunc, (void *) &pvals[i]);
18     int *rvals[4];
19     for (int i = 0; i < 4; i++) {
20         rvals[i] = malloc(sizeof(int));
21         pthread_join(hThread[i], (void **) &rvals[i]);
22         printf("Thread_%d_returns_%d\n", i, *rvals[i]);
23         //free(rvals[i]); // Segmentation fault with this line
24     }
25     return 0;
26 }
```



Contents

- 1 Warm-up
- 2 Overview
- 3 Multicore Programming
- 4 Multithreading Models
- 5 Thread Libraries
- 6 Implicit Threading**
- 7 Threading Issues



Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
 - OpenMP
 - Grand Central Dispatch (GCD)
 - Other approaches



Implicit Threading

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task
 - i.e. Tasks could be scheduled to run periodically
- Windows API supports thread pools:

```
1 DWORD WINAPI PoolFunction(AVOID Param) {  
2     /*  
3      * this function runs as a separate thread.  
4      */  
5 }
```



Contents

- ① Warm-up
- ② Overview
- ③ Multicore Programming
- ④ Multithreading Models
- ⑤ Thread Libraries
- ⑥ Implicit Threading
- ⑦ Threading Issues**



Threading Issues

- Semantics of *fork()* and *exec()* system calls
- Thread cancellation of target thread
 - Asynchronous or deferred
- Signal handling
 - Synchronous and asynchronous
- Thread-local storage
- Scheduler Activations



Threading Issues

Semantics of *fork()* and *exec()*

- If one thread in a program calls *fork()*, does the new process duplicate all threads, or is the new process single-threaded?
 - Two versions of *fork()*.
- What about *exec()*?
 - Replace the entire process, including all thread.
- Discussion: If *exec()* is called immediately after forking, which version of *fork()* do you prefer?
 - If *exec()* is called immediately after forking, then duplicating all threads is unnecessary



Threading Issues

Thread Cancellation

- Terminating a thread before it has finished
- Two general approaches:
 - **Asynchronous cancellation** terminates the target thread (i.e., the thread to be canceled) immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be canceled
- Pthread code to create and cancel a thread:

```
1 pthread_t tid;  
2  
3 /* create the thread */  
4 pthread_create(&tid, 0, worker, NULL);  
5  
6 /* cancel the thread */  
7 pthread_cancel(tid);
```



Threading Issues

Thread Cancellation (contd.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	—
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
 - Cancellation only occurs when thread reaches **cancellation point**
 - i.e., `pthread_testcancel()`
 - Then cleanup handler is invoked
- On Linux systems, thread cancellation is handled through signals



Threading Issues

Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
 - Signal is generated by particular event
 - Signal is delivered to a process
 - Signal is handled by one of two signal handlers:
 - default
 - user-defined
- Every signal has **default handler** that kernel runs when handling signal
 - **User-defined signal handler** can override default
 - For single-threaded, signal delivered to process



Threading Issues

Signal Handling (contd.)

```

1  SIGHUP          1      // Hangup (POSIX)
2  SIGINT          2      // Interrupt (ANSI)
3  SIGQUIT         3      // Quit (POSIX)
4  SIGILL          4      // Illegal instruction (ANSI)
5  SIGTRAP         5      // Trace trap (POSIX)
6  SIGABRT         6      // Abort (ANSI)
7  SIGFPE          8      // Floating-point exception (ANSI)
8  SIGKILL         9      // Kill, unblockable (POSIX)
9  SIGUSR1        10     // User-defined signal 1 (POSIX)
10 SIGSEGV        11     // Segmentation violation (ANSI)
11 SIGUSR2        12     // User-defined signal 2 (POSIX)
12 SIGPIPE        13     // Broken pipe (POSIX)
13 SIGALARM       14     // Alarm clock (POSIX)
14 SIGTERM        15     // Termination (ANSI)
15 SIGCHLD        17     // Child status has changed (POSIX)
16 SIGCONT        18     // Continue (POSIX)
17 SIGSTOP        19     // Stop, unblockable (POSIX)
18 SIGTSTP        20     // Keyboard stop (POSIX)
19 SIGTTIN        21     // Background read from tty (POSIX)
20 SIGTTOU        22     // Background write to tty (POSIX)

```



Threading Issues

Signal Handling (contd.)

- Where should a signal be delivered for multi-threaded?
 - Deliver the signal to the thread to which the signal applies
 - Deliver the signal to every thread in the process
 - Deliver the signal to certain threads in the process
 - Assign a specific thread to receive all signals for the process
- Discussion: Can a thread have its own handler?
 - All threads in a process share a same signal handler, which can be either a default one or a user-defined one (overridden).



Threading Issues

Thread-Local Storage

- How to achieve parallelism in executing *counter* ++?
 - Each thread counts, and summarize when needed.
- Is counter shared by all threads in the process, or owned by each thread?
 - Not shared within the process, also not in thread stacks.
- **Thread-local storage (TLS)** allows each thread to have its own data
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
 - Local variables visible only during single function invocation
 - TLS visible across function invocations
- Similar to static data
 - TLS is unique to each thread
- Why not use the thread stack?
 - Life cycle reasons. Local variables are in thread stack



Threading Issues

Scheduler Activations

- In M:M or Two-level model, how to maintain an appropriate number of kernel threads allocated to the application?
 - Communication, but **how?**
- Typically use an intermediate data structure between user and kernel threads — **lightweight process (LWP)**
 - Appears to be a virtual processor on which process can schedule user thread to run
 - Each LWP attached to kernel thread
 - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library. (vs. **downcalls**)
- This communication allows an application to maintain the correct number kernel threads





Threading Issues

Scheduler Activations (contd.)

- Suppose an I/O-bound application which has five different file-read requests occur simultaneously. At some moment, two kernel threads are allocated to the application.
- How many LWPs are created?
- At some other moment, one kernel thread blocks (waiting for I/O completion)
- T/F? This kernel thread makes an upcall, and the attached LWP blocks the current user thread, and schedules another user thread to run.
- T/F? The attached LWP also blocks. The kernel makes an upcall and then allocates a new LWP to the application.
- After returning from upcall handling, how many LWPs now?
- If we wish to run this application efficiently, how many LWPs are needed for this application.



Threading Issues

Scheduler Activations (contd.)

- Suppose an I/O-bound application which has five different file-read requests occur simultaneously. At some moment, two kernel threads are allocated to the application.
- How many LWPs are created? **2**
- At some other moment, one kernel thread blocks (waiting for I/O completion)
- **T/F?** This kernel thread makes an upcall, and the attached LWP blocks the current user thread, and schedules another user thread to run.
- **T/F?** The attached LWP also blocks. The kernel makes an upcall and then allocates a new LWP to the application.
- After returning from upcall handling, how many LWPs now? **3**
- If we wish to run this application efficiently, how many LWPs are needed for this application. **5**



After Class Exercise

- Exercise 4.2, p191.
 - What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?